

Backend Project Structures

Introduction

Backend applications also don't have a single perfect folder structure. The right architecture depends on your application's size, team, and business requirements.

This guide covers the most popular backend structures used in modern Node.js and Express applications.

1. MVC (Model-View-Controller)

Best For

- Beginners
- CRUD APIs
- Small projects

Folder Structure

```
src/  
├── controllers/  
├── models/  
├── routes/  
├── middleware/  
├── app.ts  
└── server.ts
```

Advantages

- Easy to understand
- Most tutorials use MVC
- Quick development

Disadvantages

- Business logic often ends up inside controllers
- Difficult to scale

2. MVC + Services Structure

Best For

- Production APIs
- Medium to large applications
- Team projects

Folder Structure

```
src/  
├── controllers/  
├── services/  
├── repositories/  
├── models/  
├── routes/  
├── validators/  
├── middleware/  
├── config/  
└── utils/
```

Request Flow

```
Route  
↓  
Controller  
↓  
Service  
↓  
Repository  
↓  
Database
```

Advantages

- Clean separation of responsibilities
- Easier testing
- Better maintainability
- Production-ready

Disadvantages

- Slightly more boilerplate
-

3. Feature-Based Structure

Best For

- Large applications
- SaaS products
- Scalable APIs

Folder Structure

```
src/
├── modules/
│   ├── auth/
│   ├── users/
│   ├── products/
│   └── orders/
```

Example Module

```
users/
├── controller.ts
├── service.ts
├── repository.ts
├── routes.ts
├── model.ts
└── validation.ts
```

Advantages

- Related files stay together
- Easier collaboration
- Very scalable

Disadvantages

- Slightly harder for beginners
-

4. Clean Architecture

Best For

- Enterprise software
- Large companies
- Complex business logic

Folder Structure

```
src/  
├── domain/  
├── application/  
├── infrastructure/  
└── interfaces/
```

Advantages

- Business logic is independent
- Highly testable
- Easy to maintain

Disadvantages

- More complex
 - Takes longer to build
-

5. Microservices Architecture

Best For

- Very large applications
- Multiple teams
- High traffic systems

Folder Structure

```
services/  
├── auth-service/  
└── payment-service/
```

```
|— notification-service/  
|— user-service/
```

Each service has its own:

- Database
- API
- Deployment
- Business logic

Advantages

- Independent deployment
- Better scalability
- Fault isolation

Disadvantages

- Complex infrastructure
- Harder communication between services

Which Structure Should You Choose?

Project Type	Recommended Structure
Beginner API	MVC
Portfolio Backend	MVC
Production API	MVC + Services
Large SaaS	Feature-Based

Enterprise Application	Clean Architecture
Massive Scale Application	Microservices

Final Recommendation

For learning backend development, start with MVC.

Once you're building production applications, move to MVC + Services, as it provides a clean separation between controllers and business logic.

Feature-Based Structure is an excellent choice for scalable APIs, while Clean Architecture and Microservices are best suited for enterprise-level applications with complex requirements.